ЛАБОРАТОРНА РОБОТА № 4

Тема: Модульне програмування, низькорівневі порозрядні операції, бітові маски та командний процес розробки через Pull Request у Git

1.МЕТА РОБОТИ Опанувати принципи функціональної декомпозиції програм: проектування чистих підпрограм із чітким розділенням на прототипи (оголошення) та реалізацію (визначення). Дослідити механізм роботи автоматичних локальних змінних на стеку та використання специфікатора static для збереження внутрішнього стану функцій між послідовними викликами. Опанувати низькорівневе маніпулювання даними на рівні окремих розрядів за допомогою порозрядних операцій & , | , ^ , ~ , << , >> . Засвоїти чотири канонічні інженерні операції над апаратними регістрами мікропроцесора: встановлення, скидання, інвертування та тестування стану бітів за допомогою динамічно сформованих масок. Опанувати командний процес інтеграції коду через створення, рецензування та злиття Pull Request (PR) на платформі GitHub. Підготувати підсумкову інженерну документацію та долучити обов'язкову пояснювальну записку у форматі PDF до звітності.

ХІД РОБОТИ:

					ЖИТОМИРСЬКА ПОЛІТЕХНІКА.20.F7.11.000 –Лр.4		
Змн.	Арк.	№ докум.	Підпис	Дата			
Розроб.		Іванюченко І.Є.			Звіт з лабораторної роботи №4	Літ.	Арк. 1 / Акрушів 8
Перевір.		Лисенко М.С.					
Реценз.						ФІКТ, гр. КІ-26-1	
Н. Контр.							
Зав.каф.		Лобанчикова Н.М.					

Етап 1. Функціональна декомпозиція та збереження стану

Лістинг

11.Функція обчислення полінома . Статична змінна рахує кількість проходів графіка через нуль ( ).

```
#include <stdio.h>

double calculatePolynomial(double a, double b, double c, double x);

double calculatePolynomial(double a, double b, double c, double x)
{
    static int zeroCount = 0;

    double y = a * x * x + b * x + c;

    if (y == 0.0)
    {
        zeroCount++;
    }

    printf("Number of zero results: %d\n", zeroCount);

    return y;
}

int main(void)
{
    int n;

    double a, b, c, x, y;

    printf("Polynomial: Y = AX^2 + BX + C\n");
    printf("Enter number of calculations: ");

    if (scanf("%d", &n) != 1 || n <= 0 || n > 100)
    {
        printf("Invalid input!\n");
        return 1;
    }

    for (int i = 0; i < n; i++)
    {
        printf("\nCalculation %d\n", i + 1);

        printf("Enter A: ");
        scanf("%lf", &a);

        printf("Enter B: ");
        scanf("%lf", &b);

        printf("Enter C: ");
        scanf("%lf", &c);

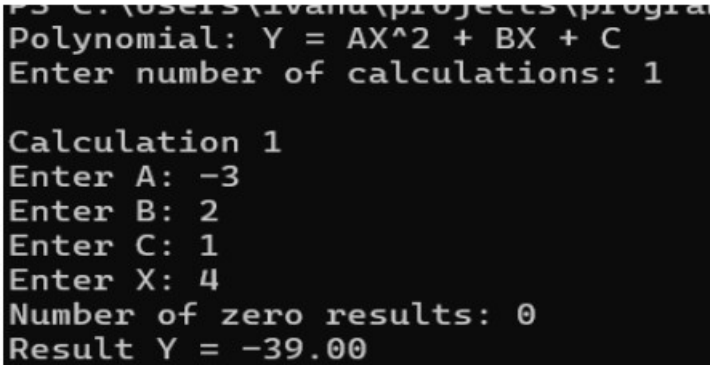
        printf("Enter X: ");
        scanf("%lf", &x);

        y = calculatePolynomial(a, b, c, x);

        printf("Result Y = %.2f\n", y);
    }

    return 0;
}
```

Перевірка:



Пояснення:

Програма обчислює значення многочлена другого степеня Y = AX² + BX + C для кількох наборів введених користувачем даних і підраховує, скільки разів результат виявився нульовим. На початку підключається бібліотека stdio.h, потрібна для введення та виведення, і оголошується прототип функції calculatePolynomial, щоб компілятор знав про неї до того, як вона буде викликана в main. Функція приймає чотири числа типу double (коефіцієнти a, b, c та значення x) і повертає результат також типу double. Усередині функції оголошено статичну локальну змінну zeroCount, яка ініціалізується нулем лише один раз, при першому виклику, і далі зберігає своє значення між усіма наступними викликами, завдяки чому працює як лічильник без використання глобальних змінних. Потім за формулою a * x * x + b * x + c обчислюється значення многочлена й записується в змінну y. Якщо воно дорівнює нулю, лічильник zeroCount збільшується на одиницю. Після цього функція виводить поточне значення лічильника й повертає результат у викликаючу функцію. У функції main спочатку виводиться заголовок з формулою, а користувача просять ввести кількість обчислень n. Введене значення перевіряється: якщо scanf не зміг прочитати ціле число, або n не додатне, або воно більше за 100, програма виводить повідомлення про некоректне введення й завершується з кодом 1. Якщо перевірка пройдена, запускається цикл, який виконується n разів. На кожній ітерації виводиться номер поточного обчислення, користувач вводить коефіцієнти A, B, C та значення X, після чого викликається функція calculatePolynomial, а отриманий результат виводиться на екран із двома знаками після коми. Коли цикл завершується, програма повертає 0, що означає успішне завершення роботи.

Етап 2. Емуляція 32-бітного апаратного регістра GPIO (task2.c)

11.Встановити біт 12; скинути біт 0; інвертувати біт 5; перевірити стан біта 22.

Лістинг

```c
#include <stdio.h>
#include <stdint.h>

uint32_t set_bit(uint32_t reg, unsigned int n);
uint32_t clear_bit(uint32_t reg, unsigned int n);
uint32_t toggle_bit(uint32_t reg, unsigned int n);
int check_bit(uint32_t reg, unsigned int n);
void print_binary(uint32_t reg);

uint32_t set_bit(uint32_t reg, unsigned int n)
{
    return reg | (1u << n);
}

uint32_t clear_bit(uint32_t reg, unsigned int n)
{
    return reg & ~(1u << n);
}

uint32_t toggle_bit(uint32_t reg, unsigned int n)
{
    return reg ^ (1u << n);
}

int check_bit(uint32_t reg, unsigned int n)
{
    return (reg & (1u << n)) != 0;
}

void print_binary(uint32_t reg)
{
    for (int i = 31; i >= 0; i--)
    {
        printf("%u", (reg >> i) & 1u);

        if (i % 4 == 0)
        {
            printf(" ");
        }
    }

    printf("\n");
}

int main(void)
{
    uint32_t reg = 0x00400021;

    printf("32-bit GPIO register emulation\n");

    printf("\nInitial register:\n");
    print_binary(reg);

    printf("\n1. Set bit 12:\n");
    printf("Before: ");
    print_binary(reg);

    reg = set_bit(reg, 12);

    printf("After:  ");
    print_binary(reg);

    printf("\n2. Clear bit 0:\n");
    printf("Before: ");
    print_binary(reg);

    reg = clear_bit(reg, 0);

    printf("After:  ");
    print_binary(reg);

    printf("\n3. Toggle bit 5:\n");
    printf("Before: ");
    print_binary(reg);

    reg = toggle_bit(reg, 5);

    printf("After:  ");
    print_binary(reg);

    printf("\n4. Check bit 22:\n");
    printf("Register: ");
    print_binary(reg);

    if (check_bit(reg, 22))
    {
        printf("Bit 22 is SET (1).\n");
    }
    else
    {
        printf("Bit 22 is CLEAR (0).\n");
    }

    return 0;
}
```

Перевірка:

```
32-bit GPIO register emulation

Initial register:
0000 0000 0100 0000 0000 0000 0010 0001

1. Set bit 12:
Before: 0000 0000 0100 0000 0000 0000 0010 0001
After:  0000 0000 0100 0000 0001 0000 0010 0001

2. Clear bit 0:
Before: 0000 0000 0100 0000 0001 0000 0010 0001
After:  0000 0000 0100 0000 0001 0000 0010 0000

3. Toggle bit 5:
Before: 0000 0000 0100 0000 0001 0000 0010 0000
After:  0000 0000 0100 0000 0001 0000 0000 0000

4. Check bit 22:
Register: 0000 0000 0100 0000 0001 0000 0000 0000
Bit 22 is SET (1).
```

Пояснення:

Програма емулює роботу 32-бітного регістра, наприклад регістра керування портами GPIO, і демонструє базові бітові операції над ним. Для цього використовується тип uint32_t з бібліотеки stdint.h, який гарантує рівно 32 біти без знака. Операції оформлено окремими функціями. Функція set_bit встановлює потрібний біт в одиницю: число 1u зсувається вліво на n позицій, утворюючи маску з одним увімкненим бітом, і вона поєднується з регістром операцією побітового АБО. Функція clear_bit скидає біт у нуль: маска інвертується операцією ~, після чого застосовується побітове І, тож вибраний біт обнуляється, а решта лишається без змін. Функція toggle_bit інвертує біт за допомогою побітового виключного АБО з маскою. Функція check_bit перевіряє стан біта: застосовує до регістра маску через побітове І й повертає 1, якщо результат не нульовий, та 0 в іншому випадку. Функція print_binary виводить регістр у двійковому вигляді: цикл іде від 31-го біта до 0-го, на кожному кроці регістр зсувається вправо, виділяється наймолодший біт і друкується, а після кожної групи з чотирьох бітів додається пробіл для зручності читання. У функції main регістр ініціалізується значенням 0x00400021, у якому спочатку встановлені біти 0, 5 і 22, і виводиться його початковий стан. Далі послідовно виконуються чотири дії, і для кожної показується стан регістра до та після зміни: встановлюється біт 12, скидається біт 0, інвертується біт 5, який був одиницею й стає нулем, а наприкінці перевіряється біт 22, і програма повідомляє, що він встановлений, оскільки жодна з попередніх операцій його не змінювала. Після цього програма завершується з кодом 0.

Етап 3. Пакування та розпакування телеметрії в бітове слово (task3.c)

11.[Біти 0..3] Версія протоколу (0..15); [Біти 4..11] Довжина корисного пакета (0..255); [Біти 12..15] Контрольна сума CRC (0..15).

Лістинг:

```c
#include <stdio.h>
#include <stdint.h>

uint16_t pack_data(unsigned int version,
                   unsigned int length,
                   unsigned int crc);

void unpack_data(uint16_t data,
                 unsigned int *version,
                 unsigned int *length,
                 unsigned int *crc);

uint16_t pack_data(unsigned int version,
                   unsigned int length,
                   unsigned int crc)
{
    return (uint16_t)(
        (version & 0x0F) |
        ((length & 0xFF) << 4) |
        ((crc & 0x0F) << 12)
    );
}

void unpack_data(uint16_t data,
                 unsigned int *version,
                 unsigned int *length,
                 unsigned int *crc)
{
    *version = data & 0x0F;
    *length = (data >> 4) & 0xFF;
    *crc = (data >> 12) & 0x0F;
}

int main(void)
{
    unsigned int version = 5;
    unsigned int length = 150;
    unsigned int crc = 10;

    unsigned int restoredVersion;
    unsigned int restoredLength;
    unsigned int restoredCrc;

    uint16_t packet;

    printf("Telemetry packet packing\n");

    printf("\nOriginal data:\n");
    printf("Protocol version: %u\n", version);
    printf("Packet length: %u\n", length);
    printf("CRC: %u\n", crc);

    packet = pack_data(version, length, crc);

    printf("\nPacked packet:\n");
    printf("HEX: 0x%04X\n", packet);

    unpack_data(packet,
                &restoredVersion,
                &restoredLength,
                &restoredCrc);

    printf("\nRestored data:\n");
    printf("Protocol version: %u\n", restoredVersion);
    printf("Packet length: %u\n", restoredLength);
    printf("CRC: %u\n", restoredCrc);

    if (version == restoredVersion &&
        length == restoredLength &&
        crc == restoredCrc)
    {
        printf("\nData verification: SUCCESS\n");
    }
    else
    {
        printf("\nData verification: FAILED\n");
    }

    return 0;
}
```

Перевірка:

```
Telemetry packet packing

Original data:
Protocol version: 5
Packet length: 150
CRC: 10

Packed packet:
HEX: 0xA965

Restored data:
Protocol version: 5
Packet length: 150
CRC: 10

Data verification: SUCCESS
```

Пояснення:

Програма демонструє пакування кількох полів різної довжини в одне 16-бітне число та їх зворотне розпакування, як це робиться, наприклад, у заголовках телеметричних пакетів. Для цього використовується тип uint16_t з бібліотеки stdint.h, який гарантує рівно 16 біт без знака. Розташування полів у слові таке: біти 0–3 займає версія протоколу (4 біти), біти 4–11 займає довжина пакета (8 бітів), а біти 12–15 займає контрольна сума CRC (4 біти). Функція pack_data збирає ці три значення в одне число. Спочатку кожне поле обрізається маскою за допомогою побітового I (0x0F залишає чотири молодші біти, 0xFF залишає вісім), щоб значення не виходило за відведену йому ширину й не псувало сусідні поля. Потім довжина зсувається вліво на 4 позиції, а CRC на 12, щоб вони опинилися у своїх бітових діапазонах, і всі три частини об'єднуються побітовим АБО. Результат приводиться до типу uint16_t. Функція unpack_data виконує зворотну дію: версія виділяється маскою 0x0F з наймолодших бітів, довжина виділяється зсувом вправо на 4 позиції та маскою 0xFF, а CRC виділяється зсувом вправо на 12 позицій та маскою 0x0F. Результати записуються за вказівниками, переданими у функцію, що дозволяє повернути одразу три значення. У функції main задаються вихідні дані: версія 5, довжина 150 і CRC 10, і вони виводяться на екран. Потім викликається pack_data, а отриманий пакет друкується в шістнадцятковому форматі, у цьому випадку це 0xA965. Далі пакет передається у unpack_data, яка відновлює три значення в окремі змінні, і вони також виводяться. Наприкінці програма порівнює початкові та відновлені значення й повідомляє, що перевірка пройшла успішно, якщо всі три збігаються, або що вона не пройшла, якщо хоча б одне відрізняється. Оскільки всі вихідні значення вміщуються у свої поля, відновлені дані збігаються з початковими, тож програма виводить SUCCESS і завершується з кодом 0.

Висновок:Під час виконання лабораторної роботи було закріплено навички роботи з функціями мови С та побітовими операціями. У першому завданні реалізовано функцію обчислення многочлена другого степеня зі статичною локальною змінною, яка зберігає кількість нульових результатів між викликами без використання глобальних змінних. У другому завданні за допомогою побітових операцій АБО, І, виключного АБО та зсувів створено функції для встановлення, скидання, інвертування й перевірки окремих бітів 32-бітного регістра, а також виведення його стану в двійковому вигляді. У третьому завданні виконано пакування кількох полів різної довжини в одне 16-бітне слово та їх зворотне розпакування з використанням масок і зсувів, що дозволяє економно зберігати дані в телеметричних пакетах. Також було використано типи фіксованої розрядності з бібліотеки stdint.h, прототипи функцій і передавання результатів через вказівники. Отримані результати підтвердили коректність роботи всіх програм і досягнення мети лабораторної роботи.